

COMP2300 Set 8 Practice Exam Solutions

Part A: Multiple Choice

- A1. C
- A2. B
- A3. B
- A4. B
- A5. C
- A6. C
- A7. B
- A8. C
- A9. B
- A10. B

Part B: Computational / Reasoning Problems

B1. Classifying dependences

- i1 -> i2: **RAW**, because i2 reads R2 produced by i1.
- i3 -> i4: **RAW**, because i4 uses the updated R6 as a base register.
- i5 -> i6: **WAW**, because both write R2.

B2. Renaming to remove false dependences

One valid answer is:

```
i1: ADD T0, R2, R3
i2: SUB T1, R4, R5
i3: ADD R6, T0, R7
i4: MUL R2, R8, R9
```

Explanation:

- i1 originally wrote R1; renaming it to T0 still lets i3 read the correct value.

- i2 writes a value that would otherwise create naming conflicts with later uses of R2; renaming it to T1 removes a false name dependence.
- The true dependence from the producer used by i3 is preserved.

Any correct renaming that removes false dependences while preserving true dataflow earns full credit.

B3. ROB allocation and retirement order

- ROB0 belongs to the oldest instruction.
- No. Finishing execution and retiring are different. Retirement must happen in program order.
- ROB0 and then ROB1 must retire before ROB2 may retire.

B4. Speculation and precise exceptions

- If younger instructions write directly into the architectural register file before the older load faults, the visible state may already contain effects of instructions that should not have committed yet. That makes the exception imprecise.
- In a ROB design, younger speculative results stay in the ROB until they retire in order. If the older load faults, the younger results can be discarded instead of having polluted the committed architectural state.

B5. Load/store reordering

- Safe, because the addresses are known to be different, so memory ordering between them does not affect program meaning.
- Not safe, because the load could observe the wrong value if it passes the store to the same address.
- Not safely reorderable yet, because the machine cannot prove non-aliasing without the older store address.

B6. OOO overlap on a cache miss

- The dependent ADD must wait, because it has a true dependence on the load result.
- The two independent arithmetic instructions and the ready younger branch may still issue in an OOO design, subject to resource availability and speculation policy.
- In a strict in-order design, the front of the machine is typically blocked by the older stalled instruction, so younger independent work cannot bypass it.

Part C: Past Exam Questions

C1. Register renaming

Register renaming assigns distinct physical / speculative names to different writes that would otherwise share the same architectural register name. This removes false dependences such as WAR and WAW, because unrelated instructions no longer appear to fight over the same register name. True dependences remain because consumers are linked to the specific producing version they actually need. This helps OOO execution by increasing the number of instructions that can proceed independently.

C2. OOO pipeline issues addressed by ARF+ROB

Four acceptable issues are:

- WAR false dependences
- WAW false dependences
- inability to recover cleanly from branch misprediction / misspeculation
- imprecise exceptions / interrupts

Other equivalent phrasings about recovery and speculative-state management are also valid.

C3. Issue queue and dynamic scheduling

The issue queue stores instructions that have been decoded but are waiting for operands or execution resources. The scheduler can select ready instructions out of order, so a long-latency miss does not necessarily block all younger independent instructions. This improves ILP and helps tolerate misses much better than a simple in-order pipeline.